

Raport Implementacyjny: Oprogramowanie Cyfrowego Sterownika Wielofazowej Przetwornicy Obniżającej

1. Wstęp

Celem projektu jest budowa dwufazowej przetwornicy obniżającej napięcie sterowanej w pełni cyfrowo za pomocą mikrokontrolera STM32G4. System realizuje sprzętową, ultraszybką regulację napięcia (Constant Voltage - CV) działającą z częstotliwością 250 kHz oraz sprzętowe ograniczenie prądu (Valley Current Limiting / Constant Current - CC). Dzięki wykorzystaniu akceleratora matematycznego (FMAC), kontrolera DMA oraz zdarzeń sprzętowych (HRTIM External Events), krytyczne pętle sterowania działają niezależnie od głównego rdzenia CPU, pozostawiając zasoby na obsługę interfejsu użytkownika (HMI) oraz funkcji pomocniczych, takich jak programowy Soft-Start czy demokratyczny podział prądów między fazami.

2. Architektura Systemu i Sprzętowy Przepływ Danych

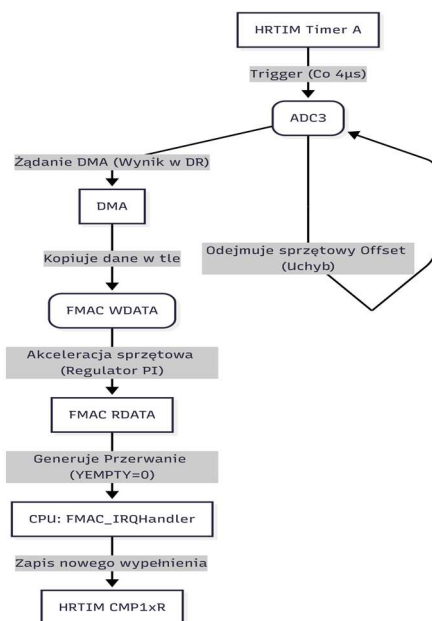
Projekt opiera się na dwóch niezależnych pętlach sprzętowych pracujących równolegle:

2.1. Główna Pętla Napięciowa (CV - 250 kHz)

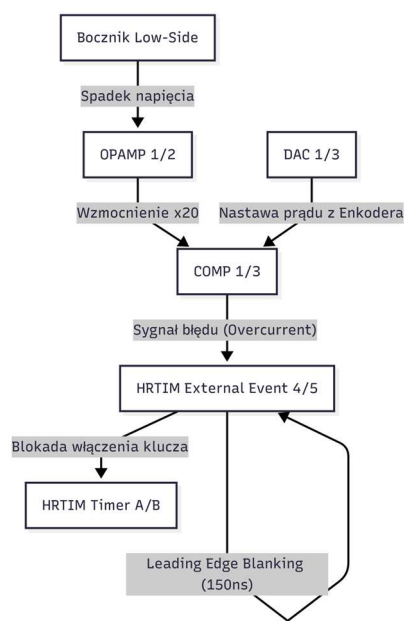
Odpowiada za stabilizację napięcia wyjściowego. Przetwarza dane w całości w domenie cyfrowej (format stałoprzecinkowy Q1.15).

2.2. Ochronna Pętla Prądowa (Valley Current Limit - CC)

Działa asynchronicznie, z opóźnieniem rzędu ~30-50 ns. Chroni dławiki przed nasyceniem i zapewnia tryb "CC", blokując kolejne cykle kluczowania na podstawie prądu fazy.



Rysunek 1 Pętla Napięciowa



Rysunek 2 Pętla Prądowa

3. Analiza Kluczowych Modułów Kodu

Poniższe fragmenty kodu ilustrują kluczowe mechanizmy systemu.

3.1. Konfiguracja Akceleratora FMAC (Regulator PI)

Akcelerator zastępuje procesor w obliczeniach filtra cyfrowego. Wyliczanie sygnału sterującego odbywa się w pełni sprzętowo.

```
/* USER CODE BEGIN 2 */
// --- KONFIGURACJA WSPÓŁCZYNNIKÓW FMAC (Regulator PI) ---
FMAC_FilterConfigTypeDef sFmacConfig = {0};

// Wartość 32767 to 1.0 w formacie Q15. Zapewnia idealną pamięć poprzedniego stanu,
// tworząc człon całkujący (I). Dzięki temu uchyb w stanie ustalonym dąży do zera.
int16_t a_coefs[2] = { 32767, 0 };

// Znaki są odwrócone (ujemny b0, dodatni b1), aby zrekompensować ujemne wartości uchybu z ADC
// (kiedy napięcie wyjściowe jest za niskie) i zapewnić ujemne sprzężenie zwrotne pętli.
int16_t b_coefs[3] = { -0x0100, 0x00F0, 0 };

sFmacConfig.pCoeffA = a_coefs;
sFmacConfig.CoeffASize = 2;
sFmacConfig.pCoeffB = b_coefs;
sFmacConfig.CoeffBSize = 3;

// Kanały DMA zasilają FMAC bezpośrednio z ADC, odciążając CPU.
// CPU interweniuje tylko na wyjściu (OutputAccess = IT), aby odebrać gotowy wynik.
sFmacConfig.Filter = FMAC_FUNC_IIR_DIRECT_FORM_1;
sFmacConfig.InputAccess = FMAC_BUFFER_ACCESS_DMA;
sFmacConfig.OutputAccess = FMAC_BUFFER_ACCESS_IT;

HAL_FMAC_FilterConfig(&hfmac, &sFmacConfig);
```

3.2. Obsługa Przerwania FMAC i Current Sharing

Najważniejsza funkcja w systemie. Posiada najwyższy priorytet sprzętowy, stąd zrezygnowano z biblioteki HAL na rzecz bezpośredniego dostępu do rejestrów.

```
void FMAC_IRQHandler(void)
{
    // Weryfikacja gotowości wyniku (YEMPTY zera, oznacza "NIE PUSTY" = gotowy wynik)
    if((FMAC->SR & FMAC_SR_YEMPTY) == 0)
    {
        // Odczyt sprzętowo kasuje żądanie przerwania i odblokowuje pipeline dla kolejnych
        próbek.
        int16_t control_effort_q15 = (int16_t)READ_REG(FMAC->RDATA);

        // BEZPIECZNE OKNO AKTUALIZACJI OFFSETU
        // Wykonywane tuż po odczycie w czasie przerwania, ponieważ mamy pewność,
        // że ADC właśnie zakończyło pomiar i flaga ADSTART = 0.
        if (update_offset_flag)
        {
            ADC3->OFR1 &= ~ADC_OFR1_OFFSET1_EN;
            ADC3->OFR1 = (ADC3->OFR1 & ~ADC_OFR1_OFFSET1_Msk) | new_adc_offset;
            ADC3->OFR1 |= ADC_OFR1_OFFSET1_EN;
            update_offset_flag = 0;
        }

        // Wyliczenie bazowego wypełnienia z dodaniem limitów nasycenia (Saturacja)
        int32_t duty_change = ((int32_t)control_effort_q15 * 21760) >> 15;
        int32_t new_ton = pwm_base + duty_change;
        if (new_ton < 435) new_ton = 435;
        if (new_ton > 10880) new_ton = 10880;

        // DEMOKRATYCZNY PODZIAŁ PRĄDU (Average Current Sharing)
        // Odczyt ciągły z ADC w locie. Odchyłka w prądach skutkuje nałożeniem delty na fazę 2
        // i odjęciem tej samej delty z fazy 1. Dzięki temu całkowita energia układu (suma
        wypełnień)
        // pozostaje stała i główna pętla napięciowa (FMAC) nie traci stabilności.
        int32_t current_ph1 = (int32_t)(ADC2->DR);
        int32_t current_ph2 = (int32_t)(ADC1->DR);
        int32_t balance_effort = (current_ph1 - current_ph2) >> 2; // Proporcjonalny błąd prądu
```

```

    if (balance_effort > 300) balance_effort = 300;
    if (balance_effort < -300) balance_effort = -300;

    // Zapis do sprzętowych rejestrów porównawczych z poprawką na prąd faz
    HRTIM1->sTimerxRegs[HRTIM_TIMERINDEX_TIMER_A].CMP1xR = new_ton - balance_effort;
    HRTIM1->sTimerxRegs[HRTIM_TIMERINDEX_TIMER_B].CMP1xR = (21760 / 2) + new_ton +
balance_effort;

    // Ucinamy ścieżkę powrotu, nie wchodząc do mało wydajnej obsługi HAL
    return;
}
}

```

3.3. Interfejs Użytkownika i Algorytm Soft-Startu

Pętla główna działa asynchronicznie jako system "czasu wolnego" (Low Priority Task). Uniknięto w niej bibliotek blokujących (opóźnień na Systicku), co zapewnia płynność HMI pomimo ogromnej liczby przerw od akceleratora FMAC w tle.

```

/* USER CODE BEGIN 3 */
if (HAL_GetTick() - last_update >= 100)
{
    last_update = HAL_GetTick();

    // 1. ODCZYT ENKODERA (BEZ PRZERWAŃ TIMERA)
    // Odczyt sprzętowego licznika zapobiega gubieniu impulsów
    int16_t encoder_delta = (int16_t) __HAL_TIM_GET_COUNTER(&htim3);
    if (encoder_delta != 0)
    {
        __HAL_TIM_SET_COUNTER(&htim3, 0);
        target_voltage += (float)encoder_delta * 0.1f;
        // (Ograniczenia safety: 1.0V do 24.0V)
    }

    // 2. SOFT-START (Bezpieczna rampa napięcia)
    // Zapobiega nasyceniu dławika potężnym prądem inrush, limitując prędkość narastania
    uchybu w FMAC.
    if (current_voltage < target_voltage) {
        current_voltage += 0.1f; // +0.1V co 100ms
        if (current_voltage > target_voltage) current_voltage = target_voltage;
    }
    else if (current_voltage > target_voltage) {
        current_voltage -= 0.1f;
        if (current_voltage < target_voltage) current_voltage = target_voltage;
    }

    // 3. WYZNACZENIE WARTOŚCI SPRZĘTOWEJ (ADC OFFSET)
    // Ograniczenie konwersji zmiennoprzecinkowych zaledwie do jednej iteracji w pętli UI.
    float v_pin = current_voltage * (33.0f / 503.0f); // Dzielnik na płytce
    uint32_t adc_val = (uint32_t)((v_pin / 2.048f) * 4096.0f);
    if (adc_val > 4095) adc_val = 4095;

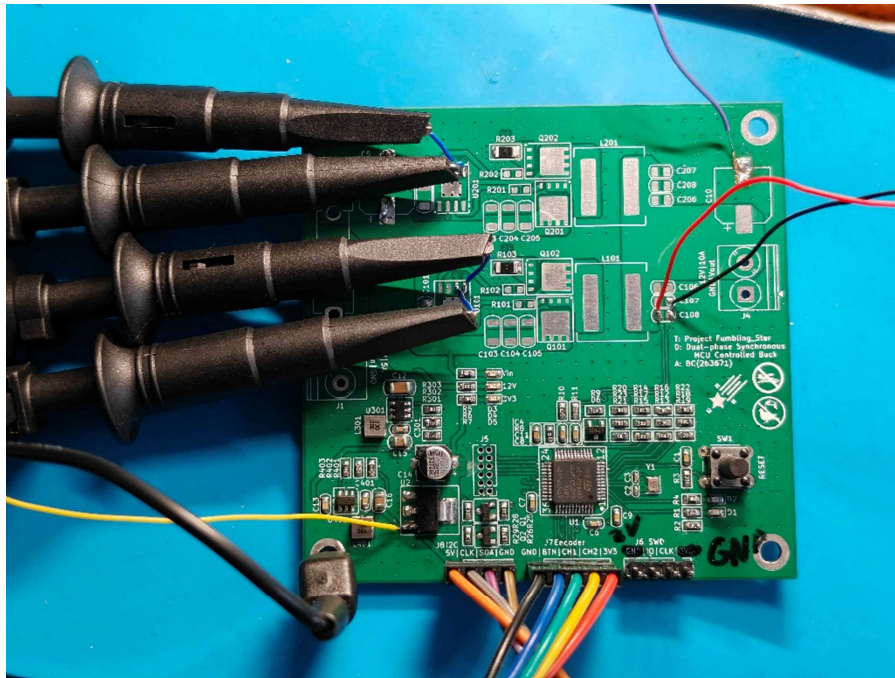
    // Przekazujemy przeliczoną zmienną do wgrania przez bezpieczne okno przerwania.
    // Dzięki temu rozwiązujemy problem "Hardware Constraint" zabraniający zapisu do
    ADSTART=1.
    new_adc_offset = adc_val;
    update_offset_flag = 1;

    // (Kod aktualizujący LCD
}
/* USER CODE END 3 */

```

4. Pomiaru weryfikacyjne

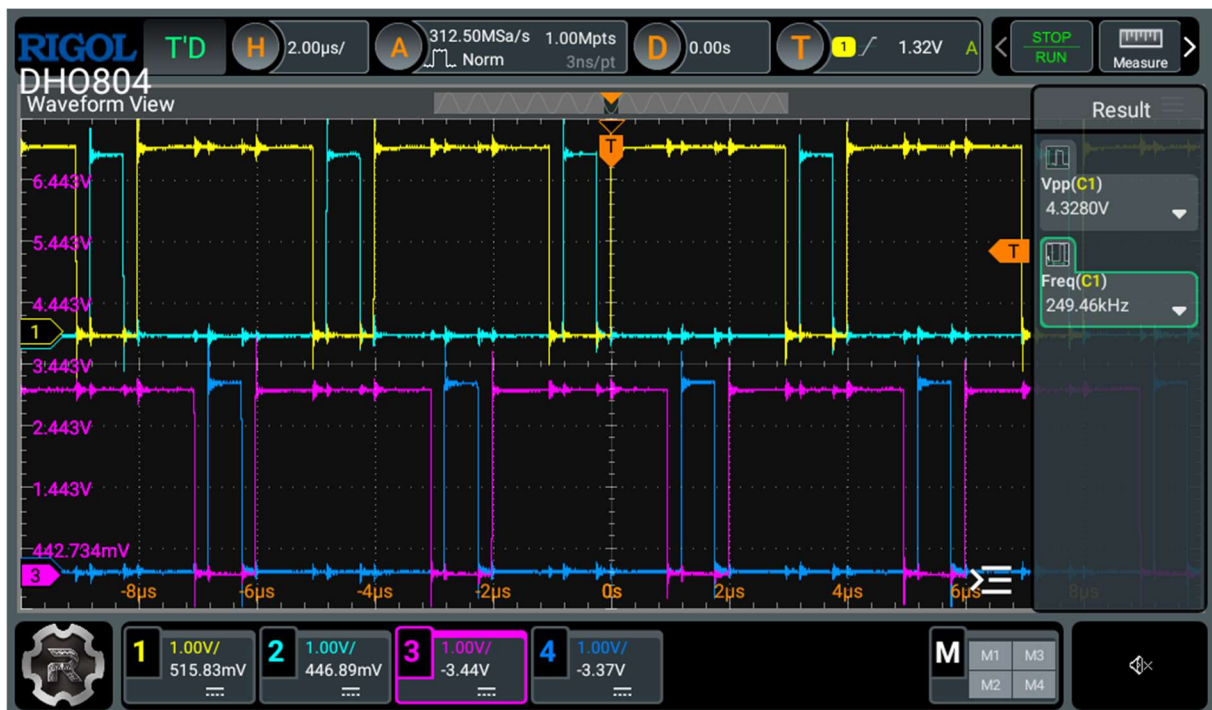
W celu weryfikacji poprawności zaimplementowania rozwiązań wykonano szereg pomiarów oscyloskopowych. Ze względu na suboptymalną konstrukcję układu pomiarowego przebiegi wykazują nadmiarowe dzwonienie i przegłos między sygnałami. Zastosowanie poprawnej techniki pomiarowej potwierdziło znaczną redukcję dzwonienia, lecz utrudniało wykonywanie pomiarów.



Rysunek 3 System pomiarowy

4.1. Przesunięcie faz

Dokonano pomiarów sygnałów na wyjściach timera HRTIM. Zaobserwowano poprawną nastawę częstotliwości, wypełnienia, przesunięcia fazowego i insercji czasu martwego



Rysunek 4 Przebiegi sygnałów na wyjściach HRTIM A i B, weryfikacja przesunięcia fazowego

4.2. Czas martwy

Włączono kanały matematyczne wykonujące operację dodawania kanałów dla poszczególnych faz. Zmieniano nastawę programową czasu martwego w celu potwierdzenia poprawnej zmiany i doboru bezpiecznej wartości. Wartość czasu martwego dla kanałów obserwowano poprzez automatyczny pomiar

szerokości funkcją -Width. Pomiar zgodny z wyliczeniami dla zegara bloku dead-time w granicach błędu powodowanego dzwonieniem.



Rysunek 5 Przebiegi sygnałów na wyjściach HRTIM A i B oraz na kanałach matematycznych, weryfikacja zmiany czasu martwego

4.3. Sprzętowe ograniczenie prądowe

Zasilacz laboratoryjny włączono w miejsce pomiaru prądu na boczniku prądowym. Zmieniano nastawę napięcia symulując przepływ prądu przez bocznicę zgodnie z zależnością $10\text{mV}=1\text{A}$. Obserwowano zachowanie kanału HRTIM B dla którego symulowano przepływ prądu. Test potwierdził poprawne działanie układu wyłączając klucz górny na czas trwania zdarzenia nadprądowego.



Rysunek 6 Przebiegi sygnałów na wyjściach HRTIM A i B, kanał B wyłączony przez przekroczenie nastawy prądu

4.4. Soft start

Zbadano zachowanie układu podczas rozruchu. W tym celu nastawiono bardzo szeroką podstawę czasu oscyloskopu i mierzono kilka pierwszych sekund startu. Zaobserwowano płynną, liniową zmianę wypełnienia.



Rysunek 7 Przebiegi sygnałów na wyjściach HRTIM A i B, moment załączenia HRTIM, wypełnienie minimalne



Rysunek 8 Przebiegi sygnałów na wyjściach HRTIM A i B, około 1,7s później, wypełnienie maksymalne przez odpięte napięciowe sprzężenie zwrotne

4.5. Balansowanie prądu faz

Programowo wprowadzono niezbalansowanie prądów faz w celu weryfikacji zmian wypełnienia między fazami. Obserwowano czas załączenia pomiarem +Width dla kanałów 2 i 4 oscyloskopu odpowiadających kluczom high side swoich faz. Zaobserwowano wydłużenie czasu załączenia dla fazy o niższym i skrócenie dla fazy o niższym prądzie.

